

Next.js continued

- **Next.js continued**
 - **What we've covered**
 - **Route Groups ()**
 - **Dynamic routes []**
 - **[...] Catch-All Segment**
 - **Catch-All [...slug]**
 - **Static Site Generation**
 - **Server side rendering V/S Client side rendering V/S Static site generation**
 - **Client side rendering**
 - **Server side rendering**
 - **Static site generation**
 - **Client components V/S Server components**
 - **When to use which ??**
 - **Some official documentation**
 - **Hydration**

What we've covered

1. Next vs React
2. CSR vs SSR
3. SEO optimisations
4. File based routing (App router)
5. "use client" directive
6. Backend routes in `next.js`
7. Data fetching in `next.js`
8. `async` components, server components

What's left

1. `()` - **Route groups**
2. `[]` - **Dynamic Segment**
3. `[...]` - **Catch-All segment**
4. Middleware in `next.js`
5. **Static site generation**
6. **Hydration**

Route Groups ()

`()` is called as **PARENTHESES** in programming

- Parentheses allow you to **create grouped routes that do not affect the URL path**.
- For example, if you have a folder named `(marketing)` or `(auth)` (Notice the parentheses attached to the name), `Next.js` will not include `(marketing)` or `(auth)` in the URL — it's just an organizational

tool to group certain routes or apply layouts without changing the URL structure.[the above example is taken from the below link]

Ref - [https://github.com/code100x/cms/tree/main/src/app/\(marketing\)](https://github.com/code100x/cms/tree/main/src/app/(marketing))

 **Remember -> To whatever the folder you put this barcket with the name, it gets IGNORED by the APP ROUTER present in `next.js`**

Why would you try to ignore the route (or folder) as isse acha bnao he mt ??

-> Below are the main reason why you should do this :-

- **Structuring** -> It helps to structure the project. If you want to put something inside the parent folder and that is not really needed to be a route then you can IGNORE them and in this case then, **The role of this comes into the picture**
- **Lower down the number of Routes / Shorten the URL** -> With this your url becomes short as you are trying to ignore some of the routes
- **Most important Reason is** -> see below and lets try to understand it by doing an Assignment

Assignment

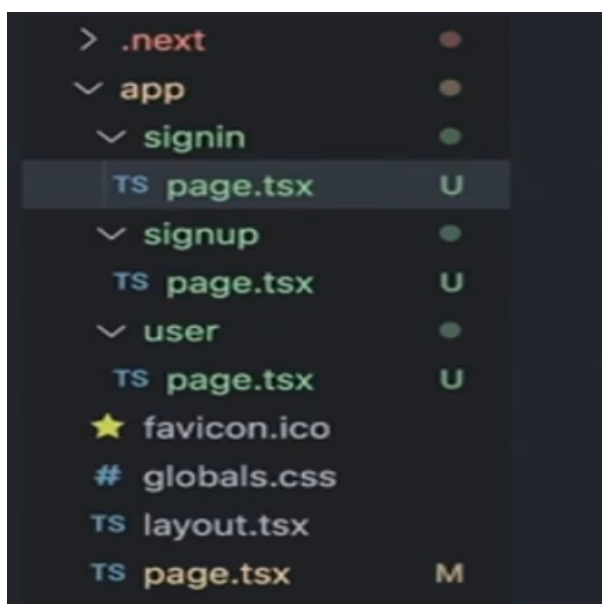
Create a `signup` and `signin` page in a `nextjs` app where both of the pages are wrapped in a layout, but no other pages are.

The website supports three pages

1. `/signup`
2. `/signin`
3. `/user`

taking the main problem statement into the consideration, you basically have to wrap the `signup` and `signin` inside the **layout**, BUT not the `user`

So from the knowledge we have gained till now, we will proceed something like this :-



1st way ->

Creating three routes(or folders) named as `signup`, `signin` and `user` and as there is already a `layout.tsx` (also called as **Root Layout as it is present inside the app folder**) that is ACTING as WRAPPING ALL THE THREE ROUTES inside one `layout` file

Now inside the `layout.tsx` file, you can apply something like the below that

```
export default function RootLayout({
  children,
}: Readonly<{
  children: React.ReactNode;
}>) {
  const route = useRoute(); // Create a custom hook that knows what the current route is
  return (
    <html lang="en">
      <body> // If the current route is "signup" or "signin", then only render the following things, for other routes (here "user"), dont do anything
        {route == "signin" || route == "signup" ? <div>header</div>}
        {children}
        {route == "signin" || route == "signup" ? <div>footer</div>}
      </body>
    </html>
  );
}
```

The above approach is very dumb way of doing the above thing, as you can clearly see that **Rootlayout is CROWDED as at last you will have a bunch of if-else statement**

2nd way ->

You can do like this -> make a separate folder (or route) and name it as `(auth)` lets say and then inside that make two folders named as `signup` and `signin` and then inside that make `layout.tsx` (this `layout.tsx` file will apply to only `signup` and `signin` as it is present inside the `(auth)` folder) and then make the `user` folder outside (i.e. -> inside the `app` folder) and you are good to go but this is slightly better way to handle the things

lets try to code the `layout.tsx` file

```
import {ReactNode} from "react"

export default function ({children} : {
  children : ReactNode // "ReactNode" basically means any component in REACT
}){
  return (
    <div>
      <div>header</div>
      {children}
      <div>footer</div>
    </div>
  )
}
```

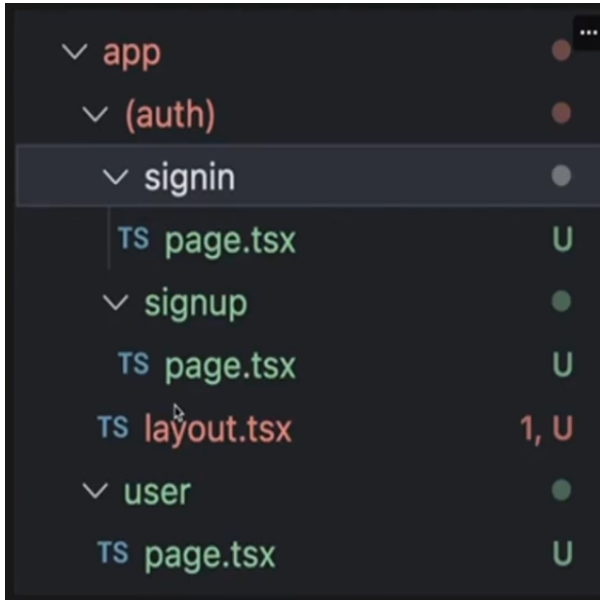
```

    )
  }

```

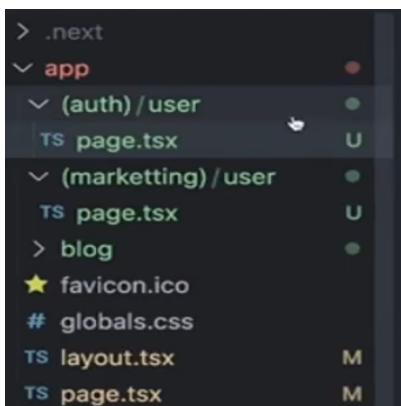
Now the above **Layout will only render on `signin` and `signup` page**

This is basically the biggest advantage of the `()` **Route Groups**, that without changing the **Routes**, you can **specifically apply the route to particular routes**



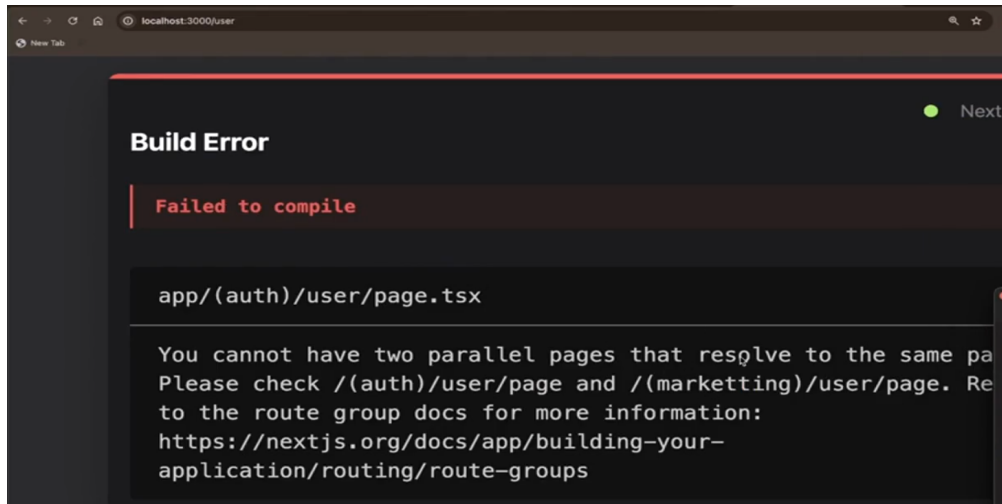
💡 **What will happen if two or more different routes groups have the same folder ex -> - `(auth)/user` and `(middleware)/user`**

-> means you have done something like this ->



Means CONFLICTING names have been given inside different folder with Route Groups `()`

In this case **YOU WILL GET AN ERROR** something like this saying that you cant have two same names for **the same route**



Dynamic routes []

A folder or file in the form `[slug]` defines a **dynamic parameter** in the route (e.g., `/blog/(slug)` might match `/blog/hello-world` or `/blog/another-post`).

Inside your components, you can access this parameter via `params.slug`.

💡 **What is slug ??** [Interview question]

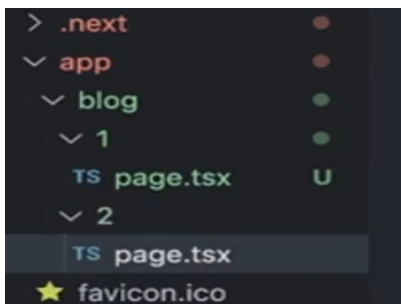
-> A **Short Hand for the blog post** For ex -> a video which has a title and your url also contains the title of the video, this will be called as **slug**

Basically a unique name for that specific page

Assignment

💡 Lets say you have to do something like this ki jb url -> "http://localhost:3000/blog/1" ho to blog 1 aa jaye and so on means jis no. blog ho wo aa jaye, How will you do this ??

-> you will say that i will do something like this ->



But the above approach is **PROBLEMATIC** as what will happen if the **3rd** page comes in and as **blogs** are fairly **DYNAMIC**, so you cant predict the no. of blogs which can come and hence **HOW MANY FOLDERS WILL YOU CREATE HERE**

so basically, the **ROUTES (or folders) are not DYNAMIC**

Assignment



Create a frontend for a `todo` / `posts` application. Backend is at

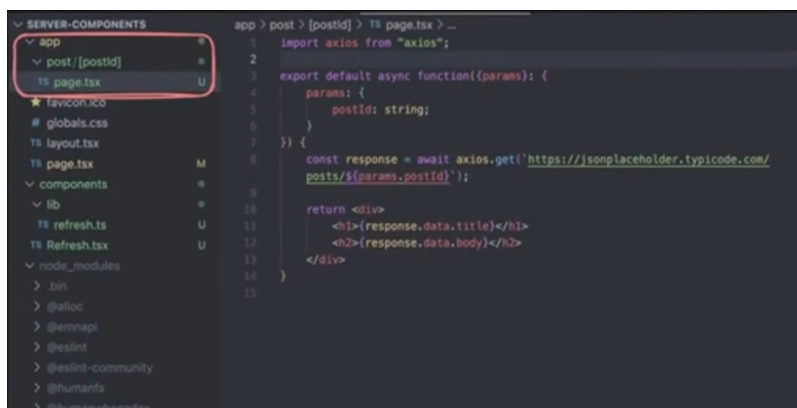
`"https://jsonplaceholder.typicode.com/posts/1"`

When the user comes to `localhost:3000/post/1`, they should see the post rendered from `https://jsonplaceholder.typicode.com/posts/1`. It **should be a server components**, feel free to use `axios`

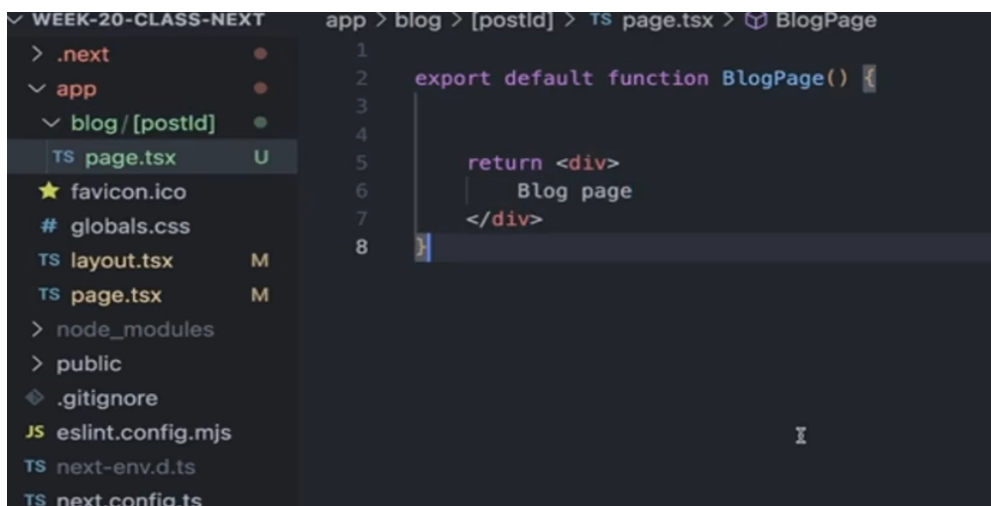
The above line simply means that when the user goes to `localhost:3000/post/1`, it should send the **backend request to `https://jsonplaceholder.typicode.com/posts/1` and the `post 1` should be rendered through the BACKEND RESPONSE came from the request**, basically we know **how to forward the request to the backend server** and we can make this but you have to make a frontend that is able to **handle the DYNAMIC parameters in the URL** so basically don't create the separate folder like the above pic instead **Create the DYNAMIC FOLDER like the below**



Remember -> To create the Dynamic Route, you have to wrap the folder inside the BIG SQUARE BRACKET `[]` and that's it you have made the DYNAMIC ROUTE

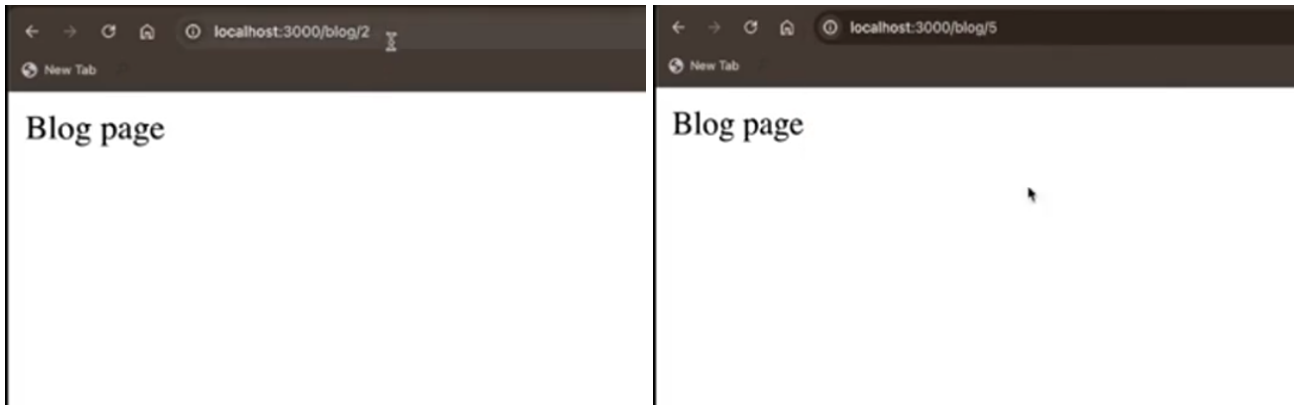


Basically let's understand what does this mean, let's understand by taking an example ->

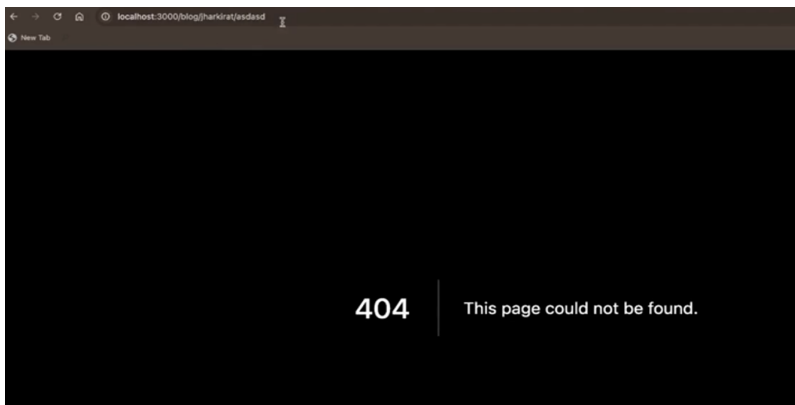


Now doing the above thing will make the **Dynamic route `[postId]` means `jo v /blog/anything_from_now` will be having this `page.tsx` applies to it, BUT remember `/blog` ke baad bas 1 route he likha hona chahiye as folder structure is like this -> `/blog/1` is CORRECT but `/blog/1/satyam` is INCORRECT**

See the output -> just notice the URL and you will notice that whether you go to `/2` or `/5`, they both have same `page.tsx` file and are rendering the same thing



But if you add something after it (i.e. after the dynamic route), then it will **not work** :-



💡 How do i get the dynamic parameter inside the file made above so that i can hit the backend and show the corresponding data on the screen ??

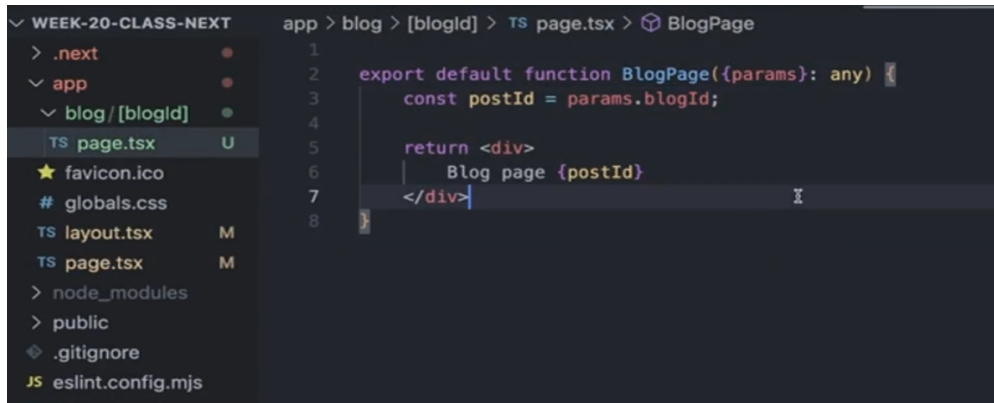
-> seeing the code

```
export default function BlogPage({params} : any){ // Give the type to "any" for
  now but figure out the better type that exists for this
  const postId = (await params).postId // this is the DYNAMIC PARAMETER made to
  ensure the DYNAMIC nature is maintained (basically jo v url me [postId] field me h
  uska value extract kiya as url h to "params" lgana pda and as "postId" naam ka
  variable me store ho rha h so "params.postId" use kiya)
  return (
    <div>
      Blog Page {postId}
    </div>
  )
}
```

📌 **Remember** -> **params** is **PROMISE** SO ALWAYS **await** it

Basically wait kr till **parameters** comes

If **[blogId]** is the name then see the below case ->



in the above pic, you have __not used `await` with the `params` so the above code will give **ERROR**.

then you would have used `const postId = (await params).blogId`.

Now **Lets connect to the backend also that according to the dynamic route, the data should be appearing corresponding to this**

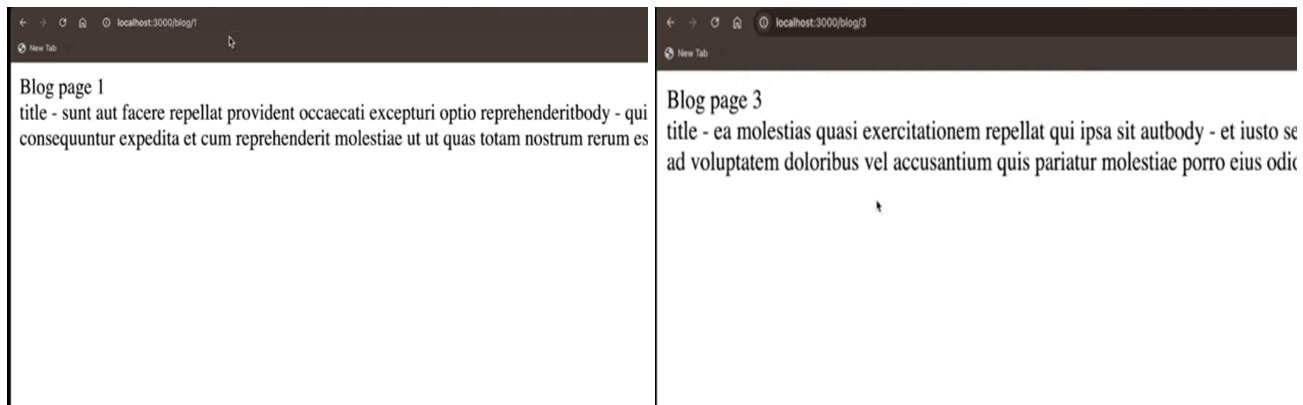
```
import axios from "axios"

export default async function BlogPage({params} : any){
  const postId = (await params).blogId
  const response = await
  axios.get(`https://jsonplaceholder.typicode.com/posts/${postId}`)
  const data = response.data // once got the data you can do anything you want
  with this

  return (
    <div>
      Blog Page {postId}
      <br />
      title - {data.title}
      body - {data.body}
    </div>
  )
}
```

- You could have also used the other way of hitting the backend or data fetching from the backend which was using `useEffect()`

Seeing the output ->



Now you can see the website is changing its content according to the DYNAMIC PARAMETER you are giving in the url and hence this is what the **DYNAMIC ROUTES []** means

The above thing is what the content loads on instagram, linkedin and other social media platform

[...] Catch-All Segment

A folder or file in the form `/docs/[...slug]` will match all segments in that position (e.g., `/docs/anything/here` will be matched by `[...slug]`).

 **Basically -> Iske baad KITNE BHI ROUTES AA JAYE tha all WILL BE HANDLED BY the `page.tsx` file made inside the folder named inside the BIG SQUARE BRACKET with the REST operator inside with the name of the folder**

Create a `app/docs/[...slug]/page.tsx`

```
export default function({ params }: {
  params: {
    slug: string[]
  }
}) {
  return <div>
    {JSON.stringify(params.slug)}
  </div>
}
```

The above thing simply means that `app/docs/ab_kitna_he_route_aa_jaye` sbme `page.tsx` [i.e - the code written above or the component which has been made or written] he lgega

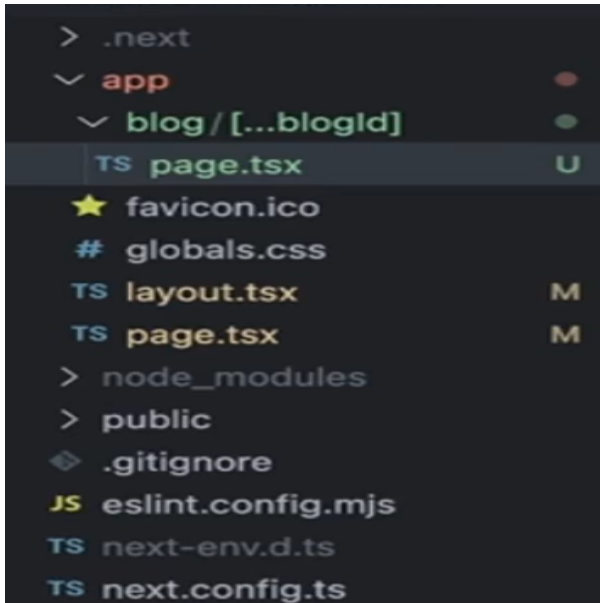
Now **of course, we have logic to render what should be there inside the component accoring to the route**

For ex -> `/app/docs/1` me jo type ka layout hoga waise sa he `/app/docs/1/34` ka v hoga and waise sa he `/app/docs/3/37/441` and so on (**bas content change rhega**)

also ONE MORE IMPORTANT THING TO NOTE THAT IT ALSO CONTAINS ALL THE CONTENTS OF THE SUB-ROUTE PRESENT BEFORE AND ON IT (that's what the Rest operator does, clones all the things which has came till now)

Let's understand by taking the same example as taken above but now the **folder structure looks like this** -

>



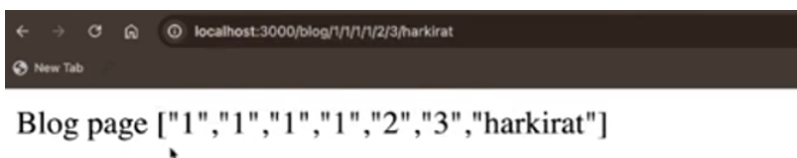
- `[blogId]` replaced by `[...blogId]`

Now inside the `page.tsx` file

```
export default async function BlogPage({params} : any){
  const postId = (await params).blogId // NOW THE "blogId" is NO MORE VARIABLE, it
  has become an ARRAY of all the routes which are attached

  return (
    <div>
      Blog Page {JSON.stringify(postId)} // as there is NO WAY TO DIRECTLY show
      the ARRAY so we have STRINGIFIED it into JSON to convert into the JSON format
    )
  )
}
```

Seeing the output ->



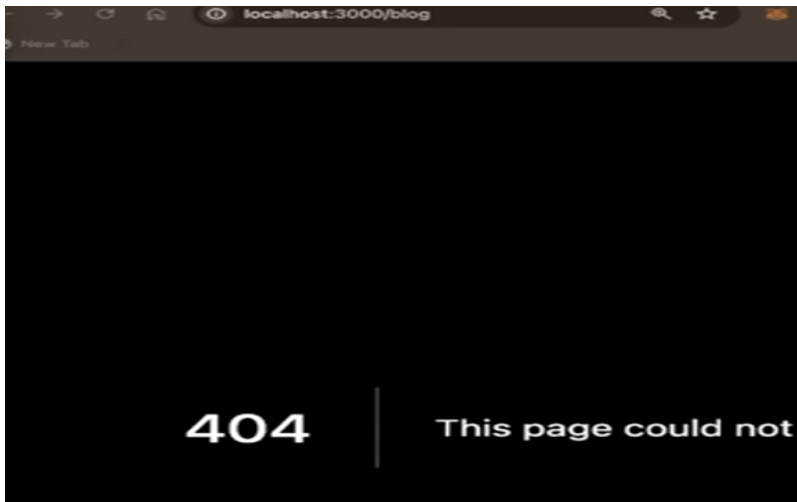
Notice the url (no matter how many routes you have given, they all are following `page.tsx` made inside the `[...blogId]` and also all the routes are being accumulated and shown (exactly what the **REST operator**

does ki **CURRENT TO STORE KRTA HE H SATH SATH PURANA BHI DIKHATA H OR SIMPLY SAYING REST JITNE H WO V DIKHAO))**

Now there is one problem in the above thing ->

it will catch all the routes **except the parent route (i.e. -> "http://localhost:3000/blog" pe ERROR de dega ye agar aise kroge to)**

you will see something like the below -> see the url



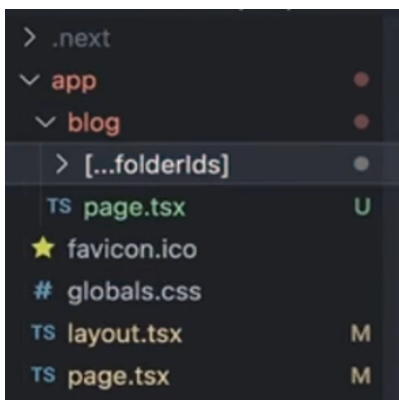
Not catching the `/blog` but will catch `/blog/anything_here`

To fix it you can do TWO THINGS ->

1st Approach ->

`blog` folder me v `page.tsx` daal do taki jb `/blog` aaye to ye `page.tsx` render ho and if `/blog/anything` (i.e. `blog` ke aage wale jitne v routes h) to jo `[...blogId]` me jo `page.tsx` h uska component render ho jaye

something like this ->>

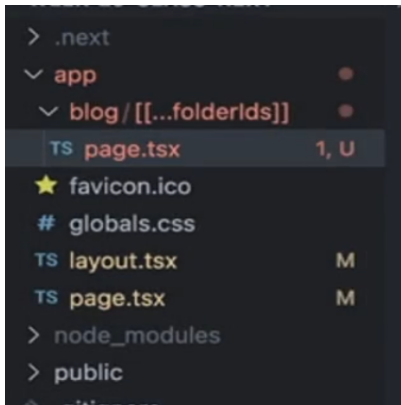


2nd Approach

Catch-All `[...slug]`

WRAP THE Catch-All routes INSIDE NOT ONLY ONE SQUARE BRACKET BUT TWO SQUARE BRACKET

something like the below ->



So instead of separate `page.tsx` for `/blog`, `/blog` will **also be handled by the same `page.tsx` made inside the `[...folderIds]`** [Notice i have just added TWO SQUARE BRACKET nothing extra done]

Although this approach is not used much, **BETTER ONE IS 1ST APPROACH only** still you should be known about this

Coming back to **Catch-All Routes** `[]`,

The **Catch-All Routes** `[]` is very useful ->

- **When you have very long route url and they all have similar component, only differ in the content**
- **You have a very NESTED STRUCTURE of your project that you have to go deeper through a chain of routes to find your content**
 - For ex -> if you go to any content management sites -> first go to the topic, then inside the subtopic, then you have the option to get video or study material and finally you get to see the video (so much nesting)

Static Site Generation

Before understanding this, lets understand why this concept exists ??

we have studied that there are two ways of data fetching from the backend,

1. making **async** component
2. using **useEffect**

Now you have the boundation of **not using the `async` component making then what will you do ??** -> you will in that case use the **`useEffect()` hook method** (as data fetching from api me yhi use hota h), then **store the value inside the STATE VARIABLE using `useState()`** and then render it using the state variable made, **BUT THIS WILL LEAD TO CLIENT SIDE RENDERING** (as **`useEffect` and `useState` CLIENT components hote h**) OR SIMPLY SAYING **react ki TARAF ZYADA CHLA GYA** instead of **Next.js** (lost the **POWER of Next.js**)

so even without defining the **async** component, we can still **achieve server side rendering HOW ??** lets see

```
import {useEffect} from "react"

// By defining the function something like this (Before Next.js 13, these two was
// being used to fetch the data as before Next.js 13, there was no such concept of
// making the component async)
// These two function only helps to achieve the Data fetching before Next.js 13
export function getServerSideProps(){ // used for Server side rendering

}
export function getStaticProps(){ // used for Static site generation

}

export default function ({params}){

  useEffect(() => {
    const response = await axios.get("api.100xdevs.com/user")
  }, [])

  return (
    <div>
      hi there
    </div>
  )
}
```

Before understanding how the static site generation is done, first lets know what are -

Server side rendering V/S Client side rendering V/S Static site generation

Client side rendering

simply means **Jo code aapke browser(CLIENT SIDE) me render ho rha h that is called as Client side rendering**

- **react** me **client side rendering** hota h

Same thing occurs in Next.js, but only on CLIENT COMPONENTs not like react(ki everything has client side rendering)

- Example of client side rendering in **Next.js** -> If you have **button** in the **Next.js** also, then it will have **client side rendering as button is CLIENT SIDE component for Next.js**

Server side rendering

The **logic(loading of HTML code or any change which needs to happen)** is **not running on the browser, it is running in the SERVER**



In the first **response** only, you got all the website rendering component required and that rendering as has happened on the server so in first **response** this got possible

Static site generation

Something which has been generated only once and NEVER CHANGE (that's what STATIC means)

OR simply saying that

If you are seeing that a particular component that looks same for everyone and present inside your codebase that can be server to everyone so WHY TO SERVER SIDE RENDER it ??

Formal definition

Whenever you build a nextjs app , some pages can be statically generated.

Static generation = HTML is built at build time and is served directly.

Take an example of a component

```
export default function Home(){
  return(
    <div>
      hi there
      <button>Click Me</button>
    </div>
  )
}
```

Now the above component if you will run in the browser, **you will notice that everytime you come to the the page where this needs to be rendered, you will see a request goes out to the Next.js server(backend part of it) where all the page content will be RENDERED and then send it to the client(browser) in response**

The problem above is that server is **RENDERING** the whole page content if everyone comes to that page and **RENDERING** is not a **FAST** or **EFFICIENT** thing to do, it takes time as well as memory(compute intensive)

So instead of the above thing, as you know that the above code is going to be **same for everyone, so dont do server side rendering when someones comes to it**

instead -> make a file **index.html** inside the **app** folder and inside that load the above code so that whenever someone comes to the site instead of server side rendering and then returning the loaded page, **index.html** code ko render kr do as **they are fast and almost have nill compute intensive and also**

Basically agar koi page me USER SPECIFIC logic nhi likha hua h to usko STATICALLY GENERATE krna is an efficient way

inside the **index.html**

```
<div>
  hi there
  <button>Click Me</button>
</div>
```

in short, **return them this STATICALLY GENERATED FILE**

and this is what known as **STATIC SITE GENERATION** means

🚩 **To make static site -> put the component which is going to be same for everyone inside the `index.html` file and you are good to go**

🚩 **STATICALLY GENERATED ARE VERY FAST WHEN IT COMES TO LOADING THE PAGE as you are loading an `HTML` with some `CSS` and `JS` only**

But you dont have to do this now, `Next.js` now is **Intelligent enough**(ensure latest version of `Next.js`) **that kaun si cheezein STATIC generate ho skti h** and then it even generate them **STATICALLY**

You can also see it by running the command

```
npm run dev
```

and then if you see the output ->

The screenshot shows the VS Code interface with the following details:

- Explorer Panel:** Shows the file structure of the project. The 'TS page.tsx' file is selected under the 'blogs/[blog]' directory.
- Main Editor:** Displays the code for the 'Blog' component. The code is as follows:


```
1 export default async function Blog({params}: {
2   params: Promise<{
3     blog: string
4   }>
5 }) {
6   // ...
7   const blogId = (await params).blog;
8   // ...
9   return <div>
```
- Terminal Panel:** Shows the output of the 'npm run dev' command. It includes a table of routes and their sizes, and a list of files generated for static site generation.

Route (app)	Size	First Load JS
/	142 B	105 kB
/not-found	979 B	106 kB
/blogs/[blog]	142 B	105 kB
+ First Load JS shared by all	105 kB	
chunks/4bd1b696-c61d11e8fea8da82.js	52.9 kB	
chunks/517-cf5517d4c8b2aea0.js	50.3 kB	
other shared chunks (total)	1.8 kB	

Below the table, the terminal shows the following output:

```
o (Static) prerendered as static content
f (Dynamic) server-rendered on demand
```

You can clearly see that `Next.js` compiler has showed us which routes are statically generated (notice the **Gola made over that route**) and **f** over that route which are **Dynamically generated**[even in the terminal it is written], as the `[blog]` is dynamically generated and changes according to the value it will get hence it has not been statically generated

🚩 You can also find the code for `HTML` file made for static site generation inside the `.next > server > app` and inside this which ever folder is statically generated by `next.js` or you that folder will have an `.html` file with the corresponding name for the folder

Client components V/S Server components

There are some of the components **which will only be rendered on the server**, and not RE-RENDER on the **client** [VERY VERY IMPORTANT LINE]

Lets try to understand this by example ->

Lets make two routes(or folders) -> **good** and **bad**(inside **app** folder) and inside their corresponding **page.tsx**

Now inside the **good**, **page.tsx**

```
export default function(){
  return (
    <div>
      hi there
    </div>
  )
}
```

and inside the **bad**, **page.tsx**

```
import {useState} from "react"

export default function(){
  const [count, setCount] = useState(0)

  return (
    <div>
      hello
      <button onClick = {() => {
        setCount(count => count + 1)
      }}>Click me! (`count is ${count}`)</button>
    </div>
  )
}
```

Now notice the **difference between good and bad page.tsx**, can you see **page.tsx of good page WILL NOT CHANGE** ("hi there" se "hi satyam" nhi ho jayega), but the code inside the **page.tsx of bad page CAN CHANGE IN THE FUTURE** (as there is state variable which will change if the button will be clicked) **which WILL RE-RENDER** and hence you will some change in the page

If you even go to the **bad**, **page.tsx** "http://localhost:3000/bad" you will clearly see an error saying as **BUILD ERROR** and hence if you want to make this code work, you have to use **"use client"** at the top of your code as IT NOW HAS BECOME CLIENT COMPONENT

```
"use client" // adding "use client" at the top of the code
```

```
import {useState} from "react"

export default function(){
  const [count, setCount] = useState(0)

  return (
    <div>
      hello
      <button onClick = {() => {
        setCount(count => count + 1)
      }}>Click me! (`count is ${count}`)</button>
    </div>
  )
}
```



Remember -> The thing which will be RE-RENDERED(or CHANGES over the time) on the screen will be made CLIENT component and The things which DONT CHANGE over the time KEEP them as SERVER components

simply saying, if a component is dependent on the **client** to make change on the page, make it CLIENT component and which does not then automatically **Next.js** will make it SERVER component

To make a component or code CLIENT COMPONENT, just add "use client" line of code at the TOP of your code and that's it now it will be CLIENT COMPONENT and will render on the CLIENT side instead of SERVER side

This was made because of the fact that some component do not change over the time so make them SERVER components and which do change over time, make them CLIENT component

Important Question



Does the statement Client components are rendered on the CLIENT side and Server component are rendered on the SERVER side is TRUE or NOT ??

-> The answer to the above question is **NO**, as **Even a Client component also renders on the SERVER side only and so is Server component**(the question statement is not the difference between them)

When to use which ??

Server components

- if you are using an **async component**
- **Want to render other server or client components**
- **Hooks that will specifically run on the server**

Client components

- if you are using **useState hook**
- if you are using **useEffect hook**

- using any **Browser-side apis** (ex -> **Window.pc**, **RTCPeerConnection** etc..)[which basically runs in **your browser only**]
- using **Custom hooks that involve use of useState or useEffect or browser-only apis**
- using **Button handlers or basically EVENT HANDLERS**

Brief documentation of it is present in the below pic

Server Components: As a general rule, Server Components run once per request on the server, so they don't have state and can't use features that only exist on the client. Specifically, Server Components:

- ❌ **May not use state**, because they execute (conceptually) only once per request, on the server. So `useState()` and `useReducer()` are not supported.
- ❌ **May not use rendering lifecycle (effects)**. So `useEffect()` and `useLayoutEffect()` are not supported.
- ❌ **May not use browser-only APIs** such as the DOM (unless you polyfill them on the server).
- ❌ **May not use custom hooks** that depend on state or effects, or utility functions that depend on browser-only APIs.
- ✅ **May use `async / await`** with server-only data sources such as databases, internal (micro)services, filesystems, etc.
- ✅ **May render other Server Components**, native elements (`div`, `span`, etc), or Client Components.

Server Hooks/Utilities: Developers may also create custom hooks or utility libraries that are designed for the server. All of the rules for Server Components apply. For example, one use-case for server hooks is to provide helpers for accessing server-side data sources.

Client Components: These are standard React components, so all the rules you're used to apply. The main new rules to consider are what they can't do with respect to Server Components. Client Components:

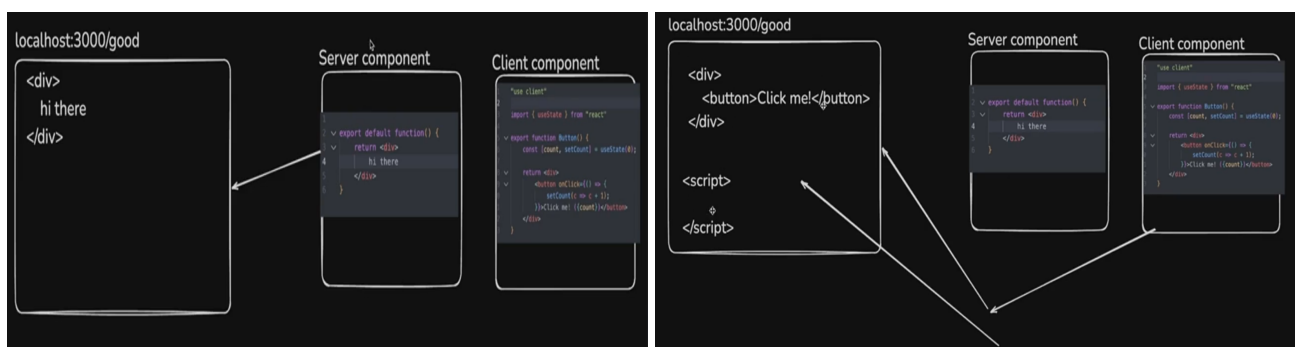
- ❌ **May not import Server Components** or call server hooks/utilities, because those only work on the server.
- However, a Server Component may pass another Server Component as a child to a Client Component: `<ServerTabContent /></ClientTabBar>`. From the Client Component's perspective, its child will be an already rendered tree, such as the `ServerTabContent` output. This means that Server and Client components can be nested and interleaved in the tree at any level.
- ❌ **May not use server-only data sources.**
- ✅ **May use state.**
- ✅ **May use effects.**
- ✅ **May use browser-only APIs.**
- ✅ **May use custom hooks and utilities** that use state, effects or browser-only APIs.

💡 Its okay that we cant make everything **Server** component but then why we cant make everything **Client** component then ??

-> Reason for **making most of the thing as SERVER Component** instead of client component is that ->

Server component are very **OPTIMAL** due to the fact that they return **HTML** file[They are almost like the static page(although they are not)] which is **NOT** the case with **Client component** as they return **HTML** as well as **JS** which also needs to render the **JS** code(some code which will make variation like button pe onClick handler lga h to wo chlna chahiye) and hence becomes slightly **UNOPTIMAL** (means **DYNAMIC** h isliye **UNOPTIMAL** hoga thoda)

A pictorial representation of what they return is also shown below :-



Notice that in the left part (server component), it is returning **HTML** code only but in right part (client component), along with **HTML**, some **JS** file is also being returned.



The above is the reason that **You should try to make as many things possible as SERVER Components**, something you cant avoid client component, there you can use but mostly you should use **Server component** and try to make them server component only

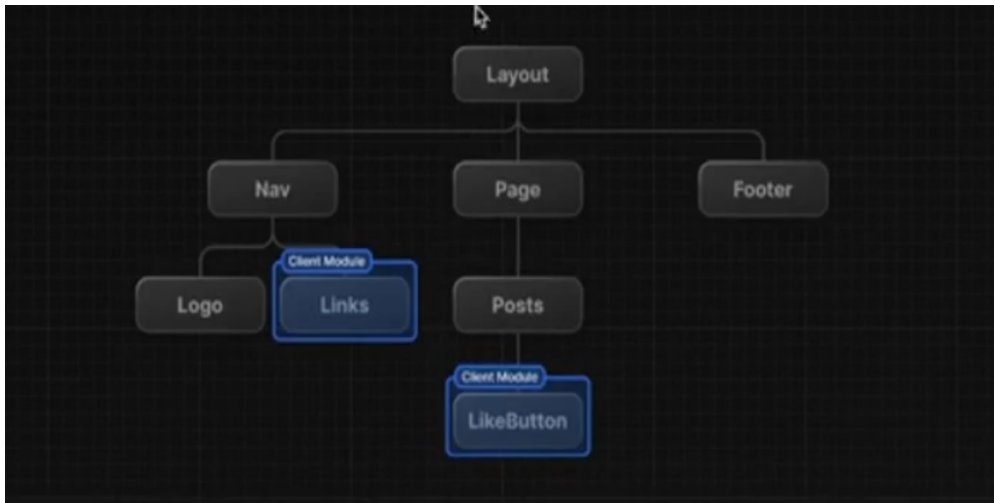
Some official documentation

The **Network Boundary** is a conceptual line that separates the Dfferent environments.

In React, choose where to place the network boundary in your component tree. For exarnple. you can fetch data and render a user's posts on the server (using Server Components). then render the interactive **Likebutton** for each post on the client (using Client Components).

Similarly, create a **Nav** component that is rendered on the server and shared across pages, but if want to show an Active state for links, you render the list Of **Links** on the client.

Your final website will look like this ->



Between the scenes, the components are split into two module graphs. The **server module graph (or tree)** contains all the **Server Components that are rendered on the server.** and **client module graph (or tree)** contains all **Client components**

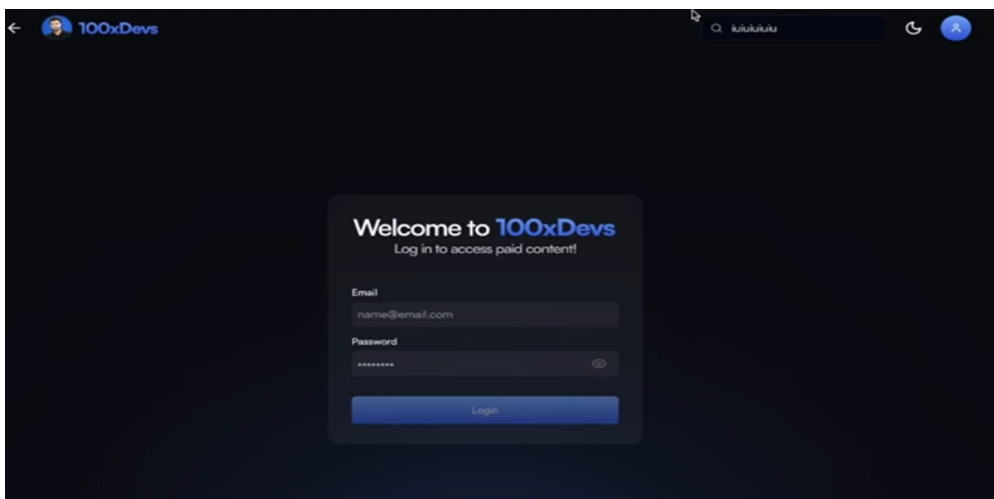
After Server are rendered, a special data format called the **React Server Component Payload (RSC)** is sent to the client.

The RSC payload contains:-

1. The rendered result of server components.
2. Placeholders (or holes) for where Client Components should be rendered and references to their JavaScript files.

React uses this information to Server and Client Components and update the DOM on the client.

Try to relate the above tree with the below page which is actually a good example of how you decide which to make server component and which to make client component



Notice in the above pic of website, the whole page is **Layout** in tree structure showed above (made **SERVER component** as **layout** thode na change hoga), inside that website also has **Navbar** which resemble to **Nav** of

the tree(also a **Server component**) and inside that **Navbar** we have logo of 100xdevs in the left which resembles **Logo** of the tree (and as this is NOT GOING TO CHANGE so made **Server component**) and the button (search, light/dark mode, profile [clickable things]) resembles **Links** of the tree (and as on **clicking these buttons, you are noticing the changes so you made it CLIENT component**) as that is only shown inside the tree (**Links** is referred to as **Client modules**)

The above thing also explains **Why you should try to avoid using Client component as much as possible (Notice in the above example, we have not made the components Client components DESPITE KNOWING THE FACT THAT YOU CAN MAKE ANY COMPONENT AS CLIENT COMPONENTs unless and untill we really needed it), till then everything was SERVER COMPONENTs**

Hydration

💡 What is Hydration in Next.js ??

The below is also the answer to **How Next.js works and why it is so optimised ??**

Hydration is the process by which a client-side JavaScript framework (such as **React**) takes over an already rendered **HTML** page and makes it interactive. In a **Next.js** application, **pages are often server-rendered (SSR) or statically generated (SSG)**. The server sends a fully formed **HTML** document to the browser, allowing users to see meaningful content quickly (which is great for SEO and performance). **Once the page arrives in the browser, React's JavaScript bundle "hydrates" that static HTML by attaching event listeners and other interactive behaviors so that the page becomes a fully functional React application.**

```
"use client"; // You can see this is made client component due to the fact that it
is using "useEffect" as well has Button handler

import axios from "axios";
import { useEffect } from "react";

export function Refresh() {

  useEffect(() => {
    axios.get("https://jsonplaceholder.typicode.com/posts/1");
  }, [])

  return <div>
    <button onClick={() => {
      console.log("onclick pressed")
    }}>click me</button>
    hello refresh component
  </div>
}
```

But if you go to the statically generated code of the above code which is present inside the **.next/server/app/doc.html**(again folder name(here **doc**) can be different as this is just for example), you will see that it has **NO INTERACTIVITY**

looks something like the below

```

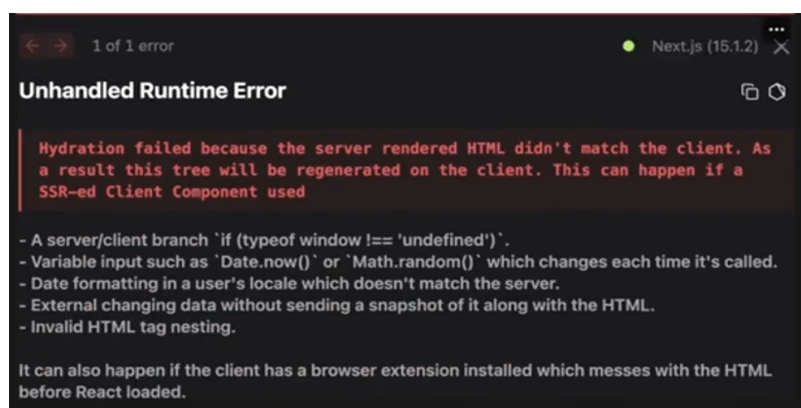
pedita et cum\nreprehender
v><button>click me</button>
ae_is" asvnc=""></script><

```

Can you notice the handler(onClick) missing from the button so **the page we rendered on the server but the interactive feature were not put in that is done by HYDRATION**

Hence the main difference between **Client component and server component** are ->

Client component are rendered on the server BUT HYDRATED ON THE CLIENT whereas in **Server component both the HYDRATION and RENDERING occurs on the SERVER**



The **Hydration ERROR** which you see time to time also happen because of the above concept only

Main Reason -> When the HYDRATION is happening on the client side and at time if the CHANGE IN VALUE has occurred between what is there when the server HTML code was sent and what has come after client hydrated it -> is what leads to HYDRATION ERROR

Very common scenario to hit into HYDRATION ERROR

A classic example of entering into the HYDRATION ERROR is **suppose you are using Date related function which has ability to give you time even (microseconds) there if you make it client component, then as it is client component so jb server pe render ho rha hoga iska code tb kuch aur time hoga and jb tk ye client pe aake hydrate hoga, tb kuch aur time hoga (microseconds ka diff. to hota he h generally) WHICH WILL EVENTUALLY LEADS TO CONFLICTING VALUES on client and server and hence HYDRATION ERROR will occur**[server and client dono pe alg-alg cheez render hui h]

in short -> **if (Server's HTML code rendered != Client's HTML code rendered), HYDRATION ERROR will occur**

That is the reason that you **whenever you are using Date.now(), you should keep it inside the useEffect hook**



as **Server IGNORES** all the things which are part of **CLIENT COMPONENTs** (such as code inside **useEffect** or **useState** or any other code which lies inside the **CLIENT component**)